



SignalForge

Private, local-first financial research on AMD Radeon

PROJECT SPECIFICATION

AMD AI DevMaster Hackathon · Track 2: Agentic AI

11

logical roles

28

deterministic tools

44/44

golden semantic gates

29.17%

end-to-end improvement

Project Summary

SignalForge is a private, local-first financial research desk for independent investors. It turns public company filings, investor-relations material, macroeconomic series, and market observations into a structured research case with cited evidence, deterministic calculations, explicit assumptions, counterarguments, and thesis-invalidating conditions.

The first complete journey compares Microsoft and NVIDIA as long-term businesses under higher-for-longer interest rates and slower AI infrastructure spending. The product helps the user decide whether a company deserves further research, belongs on a watchlist, or fits an existing portfolio thesis. It does not predict stock prices, execute trades, or issue personalized investment advice.

SignalForge is built for Track 2 of the AMD AI DevMaster Hackathon. Core inference runs locally on an AMD Radeon GPU through ROCm, with no remote model dependency in the accepted golden path.

User Scenario and Product Value

The target user is a serious independent investor who can read financial information but lacks a professional research team. Existing general-purpose chat systems can summarize documents, but they often blur reported facts, model arithmetic, assumptions, and unsupported interpretation.

SignalForge provides one coherent decision workspace:

- understand what each company does and how it earns money;
- inspect financial quality and accounting context;
- connect material economic variables through explicit transmission mechanisms;
- compare companies without erasing period, unit, or source differences;
- calculate DCF, sensitivity, reverse-DCF, multiple, beta, and statistical outputs deterministically;
- inspect citations, assumptions, calculation receipts, caveats, and counterevidence;
- ask governed follow-up questions without losing point-in-time scope or lineage;
- retain a case locally only when the user explicitly chooses to do so.

The interface uses progressive disclosure: the conclusion remains readable while evidence and calculation detail stay one interaction away.

Architecture

SignalForge architecture

Local financial research with typed authority, deterministic calculations, and specialist inference on AMD Radeon

01 INVESTOR EXPERIENCE

Research Workspace

Question, scenario controls, progressive answer, evidence drawer, calculation receipts, caveats, and governed follow-ups

Privacy controls
Inspect / export / delete

Safe projection
No prompts or CoT

02 GO CONTROL PLANE

Interpreter

Closed intent + scope

Bounded Orchestrator

Plan, fan-out, review, retry

Context Compiler

Budget, conflict, freshness

Tool / Policy Gate

Role authority + permissions

Answer Compiler

Joins, checks, publication

03 SPECIALIST TEAM · MODEL AUTHORITY

Accounting

US GAAP context

Economics

Transmission mechanisms

Valuation

Assumptions + tool calls

Market Behavior

Positioning and risk

Business Strategy

Moat and business model

Evidence Critic

Support and contradiction

Risk / Contrarian

Thesis invalidation

Final Analyst

Qualitative synthesis only

04 DETERMINISTIC AUTHORITY

Primary Evidence

SEC / IR / macro / market

28 Go Tools

Decimal finance + invariants

Receipts + Lineage

Hashes, replay, supersession

Local Case Memory

Opt-in SQLite projection

05 LOCAL INFERENCE RUNTIME

AMD Radeon GPU · ROCm 7.2.1 · gfx1100

Gemma 4 26B A4B IT QAT Q4_0 · ROCm llama.cpp · unified F16 KV · 32,768 shared context · four local workers

No remote inference

loopback-only endpoint

● Model: interpret, critique, synthesize

● Software: values, tools, policy, lineage, publication

signalforge/architecture-v1

SignalForge separates model interpretation from deterministic financial authority.

SignalForge separates model authority from software authority. Local models interpret evidence, propose qualitative claims, critique support, and synthesize a bounded semantic draft. Go owns identity, scope, evidence authorization, calculations, tool permissions, lineage, numerical relations, contract validation, and publication.

The bounded control plane has five stages:

1. The Interpreter maps the question to a closed intent and exact scope.
2. The Orchestrator creates a typed plan and fans out to at most four specialists at a time.
3. The Context Compiler retrieves primary evidence, preserves conflict, reports missing material, and enforces a context budget.
4. Independent Evidence and Risk critics disposition every releasable claim.
5. The Answer Compiler joins approved claims to evidence, receipts, and numerical references and constructs the public answer deterministically.

All logical roles share one local model server. Role-specific prompts and strict schemas provide specialization without loading many separate model weights.

Agent and Tool Capabilities

The immutable registry contains 11 logical roles covering interpretation, orchestration, accounting, economics, valuation, market behavior, business strategy, evidence criticism, contrarian risk, final analysis, and memory selection.

The product demonstrates all five Track 2 capability families:

- Local retrieval: point-in-time regulatory and investor-relations evidence with citations.
- Tool invocation: a closed, role-authorized registry of 28 deterministic financial operations.

- Multi-step planning: typed decomposition, bounded fan-out, review, and one final synthesizer.
- Multi-turn memory: parent-linked follow-ups and optional local research-case retention.
- Permission and privacy controls: read-only model authority, explicit user writes, private traces,

secret rejection, and safe UI projection.

Tool calls return immutable calculation receipts containing canonical inputs, assumptions, invariants, policy identity, code identity, provenance, and a reproducibility hash. Failed invariants cannot be represented as successful receipts.

Numerical Silence and Financial Truth

Language models are not the numerical system of record. SignalForge applies a Numerical Silence Contract across the complete workflow:

- canonical values retain exact unit, currency, fiscal period, availability, and source identity;
- deterministic Go engines perform financial, accounting, valuation, economic, and statistical

calculations;

- deterministic relations tell the model whether comparable variables increased, decreased, or

are not directly comparable;

- models receive closed symbolic references and qualitative series profiles rather than authority

to invent or modify financial values;

- the Answer Compiler renders approved numbers and citations after model synthesis.

This preserves model flexibility for interpretation while preventing a fluent answer from silently changing a value, period, direction, or formula. Decimal financial arithmetic uses cockroachdb/apd/v3 with 34-digit round-half-even policy. Statistical methods use separately declared numerical policies and independent reference checks.

Data Authority and Retrieval

The production SEC path retrieves root and historical Submissions, bounded filing documents, and Company Facts. It preserves immutable content-addressed observations, joins facts to exact filing acceptance timestamps, handles amendments, and emits point-in-time JSONL plus DuckDB and Parquet analytics.

Official investor-relations documents complement regulatory facts with history, governance, management context, presentations, and strategy. Every chunk keeps document identity, authority, issuer, publication and availability time, content hash, and supersession metadata.

The frozen retrieval evaluation contains 17 investor questions and 25 point-in-time chunks. BM25 with bounded financial-concept expansion returned each labeled evidence set with valid citations and remains the MVP production path. Semantic embeddings and Qdrant were evaluated but not added to the critical path because they did not improve the frozen complete-evidence metric.

Local AMD Radeon Deployment

The selected inference stack is:

- AMD Radeon gfx1100 with approximately 48 GiB VRAM;
- ROCm 7.2.1;
- Gemma 4 26B A4B Instruct QAT Q4_0 GGUF;
- ROCm llama.cpp with an OpenAI-compatible loopback endpoint;
- unified F16 KV cache and 32,768-token shared request capacity;

- four server slots and four product context workers;
- continuous batching and flash attention set to auto.

The model revision, tokenizer, artifact hash, quantization, runtime, ROCm version, GPU identity, dataset manifests, and run artifacts are hash-pinned. Models and downloaded source data remain outside Git. The accepted golden run used no remote inference.

Radeon Optimization Evidence

The optimization contract, workload, quality thresholds, and rollback profile were frozen before tuning. The first experiment found a reliability defect: explicit four-slot serving without unified KV split the configured 32,768-token context into four 8,192-token slot budgets. It failed all long-context cases and passed only 70 of 80 observations.

Unified F16 KV restored full shared request capacity and passed 80 of 80 isolated and concurrent contract observations. On the complete product journey, four context workers passed all 44 frozen semantic checks in 157.47 seconds. The three-worker control also passed 44 of 44 but required 222.31 seconds. The accepted path was therefore 29.17% faster end to end without lowering the frozen quality threshold.

Forced flash attention, a larger micro-batch, Q8 KV, and two-worker product concurrency were rejected by full-journey quality or efficiency gates. The rejected results remain visible rather than being discarded.

The separately recorded v57 golden decision journey completed locally in 154.33 seconds with ten model calls. It dispositioned all 42 supplied claims, released 31 authority-backed claims approved by both independent reviewers, retained complete evidence coverage, and passed all 44 checks in the pre-run frozen semantic rubric. This is bounded contract conformance, not a claim of perfect factual accuracy against external human ground truth.

Quality, Security, and Failure Behavior

The frozen adversarial matrix contains 26 cases, including 22 critical and four high-severity threats, executed through 11 repository gates. The current report records every gate passing and zero current release blockers.

The matrix covers temporal leakage, restatements, missing periods, incompatible units, receipt tampering, stale or impossible market data, conflicts, citation resolution, retrieved prompt injection, unsupported causality, direct investment instructions, guaranteed outcomes, invented evidence, malformed model output, timeout, unauthorized tools, cancellation, unsupported scope, follow-up drift, memory contamination, trace leakage, clean startup, and bounded demo load.

Failures are typed and observable. They may trigger one bounded repair, a deterministic fallback, a partial result, a clarification request, or a safe abstention. They do not silently create a publishable claim.

Memory, Privacy, and Responsible Use

Conversation continuity, durable research cases, source cache, and system telemetry are separate stores. Durable case retention is off by default. When enabled by the user, SignalForge stores only the released safe workspace projection in local SQLite with integrity hashes and restrictive file permissions.

The user can inspect, export, and delete saved cases. Model tools are read-only; durable mutation requires an explicit user action. Prompts, raw model responses, source bodies, chain-of-thought, credentials, and unbounded model context are excluded from the public projection and case store.

The final release gate rejects direct trading instructions and guaranteed, certain, or risk-free investment outcomes. SignalForge supports research judgment; it does not replace it.

Reproduction

The deterministic fixture experience runs without a GPU or model download:

```
npm --prefix web ci
npm --prefix web run build
go run ./cmd/signalforge-workspace --mode fixture --static-dir web/dist
```

The complete repository gate is:

```
scripts/verify.sh
```

It runs Go race tests, Go vet, reference-finance checks, Python tests, frontend tests and build, adversarial gates, replay validation, evidence staleness checks, and hash-bound public claim verification.

The selected Radeon runtime is reproduced through `scripts/build_llama_rocm.sh`, the hash-pinned Hugging Face model revision, `scripts/serve_llama_rocm.sh`, and the public benchmark and profiling commands documented in the repository README.

Honest Limitations

- The golden product vertical is bounded to Microsoft and NVIDIA.
- External answer accuracy has not been scored against independent human ground truth.
- Citation existence and frozen relevance do not prove arbitrary semantic entailment.
- Pattern quarantine cannot guarantee detection of novel or obfuscated prompt injection.
- Structurally plausible upstream data errors still require cross-source and human review.
- Disk encryption, multi-user authentication, and external process supervision are not claimed.
- The clean local startup gate is not a public container-pull test.
- Concurrent workspace reads do not represent unlimited concurrent 26B generation.

Evidence Index

- Repository: <https://github.com/rvbernucchi/signalforge>
- Architecture: `docs/architecture.svg`
- Golden scorecard: `evidence/golden-journey-scorecard.json`
- Safe Radeon replay: `evidence/golden-safe-decision-replay.json`
- Radeon optimization: `evidence/radeon-optimization.json`
- Adversarial matrix: `evidence/hardening-matrix.json`
- Public claim registry: `evidence/public-claims.json`
- Full evidence guide: `evidence/README.md`

All quantitative claims in this document are bounded by those public artifacts and their recorded hashes.